DPO-Mini: A Reproducible Correctness and Reference-Caching Study of Direct Preference Optimization

Technical report. Aryan Golbaghi. Public edition: 16 October 2026. Experiments completed 15 October 2026.

Abstract

We audit an independent original-sigmoid DPO implementation on SmolLM2-135M-Instruct with rank-16 LoRA, FP32 eager CPU execution, four threads, and a filtered 256/64/64 preference-pair split. The historical three-trial benchmark used five warm-up plus twenty measured updates per arm. Separate-reference versus cold-cache median training blocks were 1007.559 and 720.766 s; complete benchmark processes were 1324.106 and 1184.306 s. These boundaries differ from the single-seed, sixteen-update learning workflows (1518.458 and 1409.365 s). The new balanced three-trial comparison gives shared-base versus cold shared-cache median training times of 902.774 and 609.977 s, and whole-process times of 1147.964 and 1013.617 s. Pretrained TRL function comparisons expose an FP32 tolerance failure, traced to distinct score-reduction kernels on identical logits; diagnostic FP64 reductions recover matching adapter gradients. Shared-base reference invariants and short trajectories pass. Independent raw-score and dataset audits support the historical numerical claims. Recorded historical aggregates reconcile the total runtime difference, while finer per-phase attribution remains unresolved. These small, single-machine measurements establish scoped computational behavior, not general model quality or end-to-end TRL trainer equivalence.

1. Research questions and prior work

This study asks whether the objective and completion scores are implemented correctly, whether a fixed reference can be reused without changing the measured trajectory, and how reference residency and cache construction alter observed costs. A second uncached reference model is a useful simple baseline, but LoRA also permits a reference forward through the same frozen base with the policy adapter disabled. The follow-up therefore separates residency from caching. Neither DPO nor reference-score caching is proposed as a new method.

DPO derives a preference objective using a policy/reference log-probability ratio [R1]. The original public implementation supplies a primary implementation reference [R2]. TRL exposes DPO training and reference-log-probability precomputation [R3,R4]. LoRA represents a trainable update through low-rank matrices [R13]; the PEFT adapter API provides the disabled-adapter context used here [R12,R15]. This report distinguishes mathematical checks, executable function comparisons, and complete optimizer-loop experiments. Agreement at one level does not establish agreement at every level.

2. Objective and scoring contract

$$s_\theta(x, y) = \sum_{t=1}^{|y|} \log p_\theta(y_t \mid x, y_{<t})$$

$$m_i = (s_\theta(x_i, y_i^+) - s_0(x_i, y_i^+)) - (s_\theta(x_i, y_i^-) - s_0(x_i, y_i^-))$$

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} \log(1 + \exp(-\beta m_i)), \qquad \beta = 0.1$$

Scores are completion-token sums. The first completion token is predicted from the final prompt position; the final assistant-ending token is also scored. The generation prompt is rendered once using the pinned chat template. Each completion is suffix-encoded without added special tokens and receives exactly one assistant end token. Empty responses, embedded ending tokens and sequences above the length cap are rejected. This construction avoids assuming that a fully rendered conversation has the same token prefix as a separately rendered generation prompt [R8].

Prompt tokens remain visible to attention and can receive contextual gradients, but are excluded from direct score reduction. Padding is excluded by position, including when padding and end IDs alias. Masked invalid labels and nonfinite logits are never gathered or multiplied by zero: valid positions are selected before log-softmax. The independent scorer uses log-softmax, gather and an indexed sum; the DPO loss uses stable negative log-sigmoid. These choices define a numerical implementation, not a promise of bitwise equivalence to every algebraically equivalent kernel.

When policy and reference scores are equal, the loss is log(2), and the derivatives with respect to chosen and rejected policy scores are -0.05 and +0.05. For policy scores [-2,-4] and reference scores [-3,-3], the relative margin is 2 and the loss is softplus(-0.2). The score derivatives are not zero at initialization. Gradcheck uses float64 and a fixed reference; separate tests cover shift, causal masking, unequal lengths, prompt context and partial accumulation windows.

3. Data, model and execution methods

The model revision is 12fd25f77366fa6b3b4b768ec3050bf629380bac; the UltraFeedback Binarized revision is 3949bf5f8c17c394422ccfab0c31ea9c20bdeb85 [R6,R7]. Training and validation are drawn only from train_prefs; test rows come only from test_prefs. The retained population consists of compatible single-turn conversations with the same user context, nonempty distinct completions, and finite strictly ordered preference scores when those fields are supplied. Labels are not relabelled. This intentionally narrow population is not representative of the full dataset [R14].

Prompt groups use Unicode NFKC normalization, collapsed whitespace and casefolding. All valid official-test prompt groups, including over-length rows, are protected before selecting training data. Accepted rows are sorted by content ID, reduced to one pair per normalized prompt group and shuffled with split seed 314159. The first 64 training-pool rows form validation, the next 256 form training, and 64 eligible official-test rows form test. Both chosen and rejected formatted sequences fit 256 tokens; no completion is silently truncated. A separately written parquet audit reconstructs filtering, IDs, tokens, masks, grouping and selection and checks every retained row.

After the stated exclusions, one duplicate normalized-prompt group was removed; no training row required removal for an official-test prompt conflict. All three selected split memberships and disjoint prompt-group sets were reproduced exactly by the independent audit.

Table 1. Original filtering, independently reconstructed. Counts are sequential exclusions.

Official source	Original	Length excluded	Non-strict scores	Empty / identical	Eligible
train_prefs	61135	43133	7286	81 / 424	10211
test_prefs	2000	1360	270	3 / 12	355

The historical and follow-up experiments use Windows 11, Python 3.12.10 and an AMD Ryzen 5 PRO 7540U with six physical cores, twelve logical processors and 30.67 GiB physical RAM. Four PyTorch computation threads, FP32 weights/computation/reductions, eager attention, deterministic algorithms and evaluation-mode dropout are fixed. No CUDA device is used. Hardware exclusivity is not established; unrelated processes are not inspected. Pinned versions are torch 2.9.0, transformers 4.57.1, PEFT 0.17.1, TRL 0.24.0, accelerate 1.10.1, datasets 4.3.0 and NumPy 2.2.6. Backend and interop settings are retained per run [R9-R11].

The model has 135,436,608 parameters, of which 921,600 are trainable LoRA parameters. Rank 16, alpha 32 and zero adapter dropout apply to verified q_proj/v_proj linear targets. Identity-preserving initialization makes the initial adapter policy equivalent to the base. AdamW uses learning rate 5e-5, betas (0.9,0.999), epsilon 1e-8, zero weight decay and clipping norm 1.0. One preference pair (two sequences) is the microbatch; sixteen pairs form an optimizer update. Losses are weighted by the actual accumulation-window size, including a partial final window.

The single learning seed is 42. The sixteen-update schedule has one warm-up update: updates 1 and 2 both use 5e-5, followed by linear decay to 5e-5/15 at update 16. The 25-update timing schedule also has one optimizer warm-up update, and ends at 5e-5/24. This optimizer schedule is distinct from excluding five benchmark warm-up updates from the measured block. A prior Qwen pilot [R5] did not fit the CPU budget; it was not retried in this pass. Timing repetitions do not add learning seeds.

4. Historical audit and equality scope

The original paper, source, analysis and manifests were archived before edits. The reconstructed source archive matches the historical source hash 88d7f630bae217476e88af8e5362a82f86d3d7f139e52200e9d0c17c6fc79dae and plan 1ba6e9976791feb50acc9766f9ed6f6e731737fb7bdf3369f1809e050e22ada8. The audit verifies complete update/pair counts, the actual integer schedule, split identities and independently recomputes DPO losses and margins from saved per-example scores using Python double arithmetic. Differences from saved FP32 arithmetic stay below 1e-7. This is stronger than checking a manuscript against its own summary.

One historical cold-cache timing attempt ended after 23 of 25 total updates (18 of 20 measured updates). It has no completed metrics and is excluded from all complete-trial medians. Its exact exit cause and time are unknown; the original interruption observation, partial trace and separately identified retry remain available. The previous budget charges an upper elapsed-time allowance rather than assuming the missing interval was free. No failed attempt is reclassified as a completed observation.

The two historical learning checkpoint files have identical SHA-256 values and their complete saved fields were compared: model tensors, optimizer state, scheduler, scaler placeholder, update/data position, trace and RNG state. Thus byte identity is supported for those particular files. It is not inferred from a model hash. The old model hash included names and bytes but did not explicitly encode shapes or dtypes. The new canonical digest sorts names, includes shape and dtype and length-delimited contiguous bytes, and includes tied aliases. Shared-base checks additionally cover all named buffers, including nonpersistent buffers. Adapter-only equality is always stated together with a separately identified unchanged base.

Table 2. Historical complete-trial medians (range); three timing trials per arm. RSS is sampled during measured training.

Historical arm	20-update training, s	Benchmark process, s	Training RSS, MiB
Separate reference	1007.559 (987.683-1032.400)	1324.106 (1291.717-1344.558)	2761.484 (2759.867-2764.324)
Cold separate-built cache	720.766 (611.058-754.247)	1184.306 (1035.392-1213.297)	2103.461 (2102.078-2104.289)
Warm existing cache	708.487 (610.038-728.726)	935.511 (812.629-942.912)	2100.984 (2093.746-2101.227)

The ratio-of-medians reduction is 28.46% for the measured training block, 10.56% for the cold benchmark process, and 7.18% for the single executed learning workflow. Throughput increase uses a different denominator: the historical training-block value is 39.79%. The benchmark contains 400 processed pairs, including 80 warm-up pairs; the measured 320 pairs contain 40,761 scored completion tokens. The learning workflow processes 256 pairs in sixteen updates with evaluations, generations and a final checkpoint.

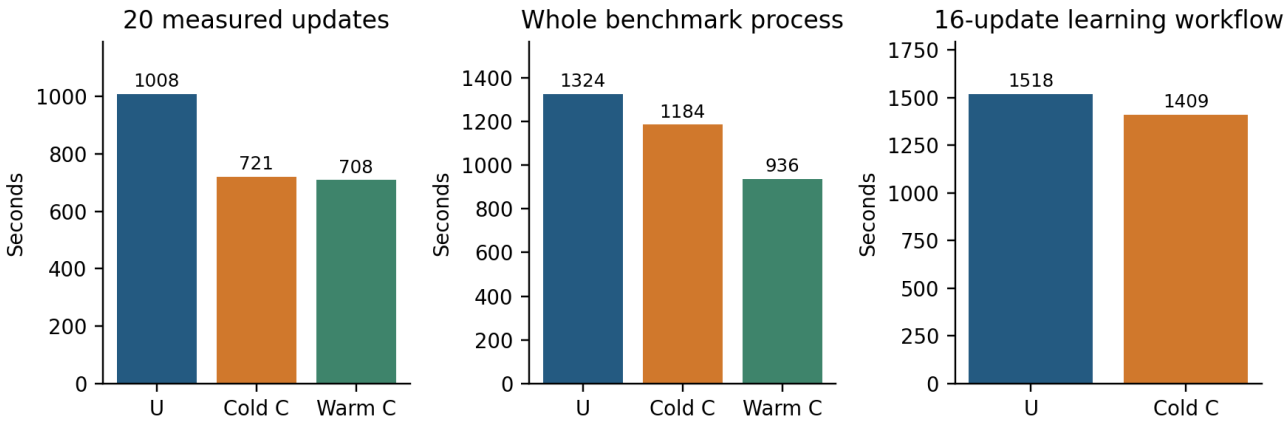


Figure 1. Three historical timing boundaries, deliberately separated. Learning bars are single executions, not three-trial medians. Cold cache includes construction; warm cache excludes an already paid construction.

5. Correctness and the pretrained upstream discrepancy

The final working core suite contains 19 passing tests, with 0 failures, 0 errors and 0 skips in its successful run. Added cases check atomic checkpoint failure preservation, canonical identity scope and adapter-context restoration after exceptions. The first runner attempt had four fixture-setup errors from a missing temporary parent directory; fixing the runner made the complete suite pass. This was not a model-correctness failure. Offline collection does not download models. Pretrained integrations remain separate commands.

The installed TRL comparison executes the real 0.24.0 concatenated_forward and original sigmoid loss functions with reverse KL, beta 0.1, zero label smoothing, no reference synchronization, no weighting, no auxiliary/RPO losses and no alternative length normalization. The same detached reference-score tensor, computed locally from the frozen reference, enters both loss functions to isolate policy-scoring and objective arithmetic. Four fixed training rows are selected by preserved order before scoring, with 4-130 completion tokens. One-pair batches match production padding and positional handling. Every named trainable LoRA gradient and its presence pattern is checked. The second fixture uses two deterministic training-only updates; its adapter tensors and outputs change. A deliberately changed completion token is detected by the negative control.

Table 3. Maximum absolute differences. Original pretrained thresholds were exceeded and are retained as failures; diagnostics are a separate scope.

Comparison	Score error	Loss error	Gradient error	Interpretation
FP64 score loss / TRL	n/a	0	0	Original bounds pass
Historical tiny GPT-2 / TRL	0	0	2.2352e-8	Tiny function scope
Pretrained initial, FP32	6.1035e-04	1.5259e-05	1.9977e-06	Original score/loss bounds fail
Pretrained trained, FP32	6.4087e-04	1.6809e-05	2.0103e-06	Original score/loss/gradient bounds fail
Initial, FP64 reduction diagnostic	1.7053e-13	9.992e-15	0	FP32 weights/logits, FP64 reducers
Trained, FP64 reduction diagnostic	1.9895e-13	6.5503e-15	0	FP32 weights/logits, FP64 reducers
Shared reference / separate	0	0	n/a	Before and after adapter updates

The starting absolute bounds were 2e-5 for FP32 summed scores, 2e-6 for FP32 gradients, 1e-12 for float64 score loss and 1e-11 for float64 score gradients; an FP32 loss bound of 2e-6 was also declared. The trained-adapter gradient error exceeds 2e-6 by about 1.03e-8. The score

discrepancy is not a tokenization mismatch: observed IDs, attention masks and model logits are bit-identical. A selected-token arithmetic diagnostic compares the local CPU log-softmax and installed TRL selected-logit-minus-logsumexp routines on a tokens-by-vocabulary matrix: they differ by up to 1.775e-5 per token in FP32 and at most 3.09e-14 in FP64. That diagnostic uses a different tensor shape from the full TRL forward. A separate full-forward diagnostic casts returned FP32 logits to FP64 while retaining both actual scoring methods; parameter gradients then match exactly for both fixtures.

One post-failure numerical adjudication uses a length-aware score bound N*2e-5 plus the standard FP32 summation term gamma_(N-1) times the absolute score sum, and a 1-Lipschitz softplus loss bound. A single empirical FP32 gradient allowance of 4e-6 is recorded, with its revised status explicit. This does not turn the original threshold failures into passes or establish a general error guarantee. Production scoring code and examples were not changed to make agreement look better. The result supports scoped numerical consistency of the implemented objective; complete TRL trainer equivalence is untested.

6. Shared-base reference and new trial design

The shared reference runs the fixed pretrained base with the current LoRA adapter disabled through the inspected PEFT 0.17.1 implementation. The cited API documentation is version 0.17.0, a distinction recorded rather than hidden [R15]. The wrapper rejects merged adapters and trainable base parameters, runs reference inference without gradients, and restores adapter enablement, active adapters, per-module train/eval flags, requires_grad flags and existing gradient objects on normal and exceptional exits. Policy training retains its gradient path. This equivalence depends on an initially identity adapter and a base reference; disabling a policy adapter would not automatically reproduce a different trained reference adapter.

All named base parameters and all buffers remain unchanged after actual pretrained updates; optimizer parameter identities are exactly the intended adapter identities. Disabled-adapter scores match a separately resident frozen reference before and after nontrivial updates. Three-update matched trajectories (six pairs per strategy) produce identical loss traces and adapter tensors for separate reference, shared base and cached shared base. A second smoke check exercises the actual instrumented worker in fresh processes. The new optimizer loop computes reference scores before creating the policy backward graph; the original-arm accounting replay preserves the historical ordering and cache construction.

The follow-up plan was frozen locally after inspecting historical results, the numerical diagnosis and short feasibility checks. It is a new protocol, not an untouched confirmatory study. Three fresh-process trials per arm use 5 excluded warm-up updates plus 20 measured updates, with sixteen pairs per update. The 256 training pairs are shuffled once with seed 42 and cycled to 400 processed pairs; the measured block covers positions 81-400, so processed-pair counts include repeated examples. Order is ABC, BCA, CAB. Each arm restores the same saved initial adapter/empty optimizer state and RNG state against an independently hashed base. Model state, data order, precision, schedule, progress logging and CPU thread settings are matched. Reference preparation cannot advance the restored training RNG.

A keeps a policy and separate frozen reference resident. B keeps one base with a trainable adapter and recomputes reference scores with that adapter disabled. C constructs a new train-only score cache through B inside every trial process, persists and verifies it, and performs zero reference forwards during training. A versus B measures residency/implementation strategy; B versus C isolates caching against the stronger baseline; A versus C combines both changes. Historical cold C used a separately constructed reference and is a different cohort. No old and new timing trials are pooled.

After timed training, every new timing arm saves the same small adapter endpoint for named-tensor verification. This I/O and final identity checks are included in whole-process time and excluded from the measured training block. These endpoint files are not full learning-workflow checkpoints, and no held-out evaluation or generation is performed in the A/B/C timing processes.

7. New timing and memory results

Table 4. New A/B/C median (range), three trials each; 320 measured pairs after 80 warm-up pairs. Whole processes include all cold setup.

Arm	Training, s	Whole process, s	Training RSS, MiB
A: separate reference	901.831 (897.240-902.195)	1151.478 (1145.515-1159.711)	2632.430 (2623.488-2637.246)
B: shared base	902.774 (896.044-907.790)	1147.964 (1142.434-1159.711)	2098.902 (2098.332-2102.840)
C: cold shared cache	609.977 (609.702-611.667)	1013.617 (1013.588-1014.831)	2095.672 (2095.641-2101.301)

A: separate reference to B: shared base has ratio-of-medians runtime reductions of -0.10% for measured training and 0.31% for the whole cold benchmark process.

B: shared base to C: cold shared cache has ratio-of-medians runtime reductions of 32.43% for measured training and 11.70% for the whole cold benchmark process.

A: separate reference to C: cold shared cache has ratio-of-medians runtime reductions of 32.36% for measured training and 11.97% for the whole cold benchmark process.

Negative reduction means the comparison took longer. These are descriptive measurements on this machine, without a statistical-significance claim.

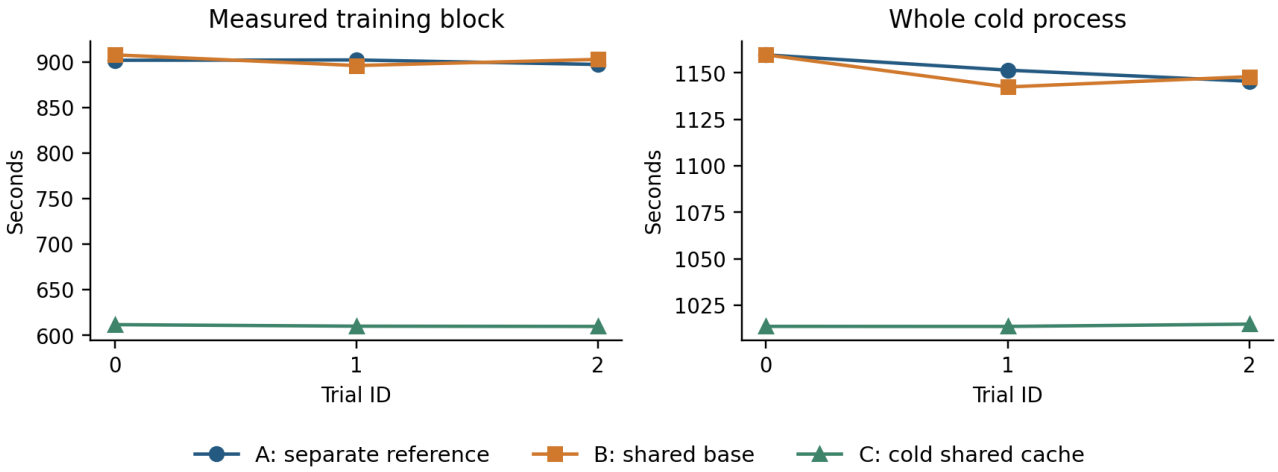


Figure 2. New per-trial training and whole-process durations. Every completed trial is shown. Different arm order balances one simple order effect; three trials cannot characterize all system variability.

Training sampled RSS medians are 2632.4, 2098.9 and 2095.7 MiB for A, B and C. Whole-process OS peak working-set medians, including cold construction, are 2739.2, 2199.7 and 2113.0 MiB. C pays a measured train-cache precompute median of 228.976 s. Whole-process peak memory is not described as cache-only memory, and comparisons never substitute OS peaks for training RSS.

The stronger baseline changes the interpretation of memory savings. The difference between A and C medians decomposes into 533.5 MiB from A minus B and 3.2 MiB from B minus C. A-to-C alone therefore does not identify a cache-specific memory benefit. B-to-C is the relevant caching comparison when an identity-initialized adapter shares a frozen base. A-to-B also includes the disabled-adapter implementation overhead; these descriptive contrasts do not separate every low-level mechanism. Cold-process results, rather than the favorable training block alone, determine the observed benefit at this fixed work count.

All nine benchmark endpoints have matching canonical full model-state hashes: True. Complete update traces match: True. These timing endpoints are not additional held-out learning checkpoints. Memory is sampled RSS at 10 ms intervals, including resident models; it can miss brief peaks. OS whole-process peak working set is a separate metric retained in raw records.

GPU memory is unavailable, not recorded as zero. The cold-cache construction stage remains in process time and whole-process memory even if its measured training block is cheaper.

8. Complete-workflow accounting and amortization

The historical twenty-update medians project to U=806.048 s and C=576.613 s per 256-pair training pass, with train-cache precomputation P=236.940 s. Under equal other setup, one pass makes caching 7.505 s slower and breaks even after 1.033 passes. In contrast, the actual historical learning workflows differ by 109.093 s in favor of caching. The projection excludes held-out evaluations, generations and checkpoint writing and assumes stationary per-pair timing. It therefore does not predict the complete executed workflow.

The recorded historical aggregate stages do reconcile the observed workflow difference: cached setup costs 379.740 s more, training costs 248.925 s less, combined evaluation/generation costs 238.339 s less, and checkpoint writing costs 1.155 s less. The residual cached-minus-uncached difference is -0.415 s. These signed components sum to the observed 109.093 s saving. They are recorded aggregates, not invented per-split or per-phase attribution. In particular, historical learning precomputed 384 pairs across three splits, whereas the benchmark projection uses a 256-pair train cache.

Old records lump evaluation and generation together and do not split cache construction by dataset split. This leaves finer historical per-phase attribution unresolved despite the aggregate reconciliation. The new replay uses the original sixteen-update protocol and original separate-reference cache design, including train/validation/test precomputation for cached learning. It measures nonoverlapping monotonic stages and a separate parent Popen-to-exit duration. Residual time is the measured difference between the parent duration and allocated stages; it includes imports, launch, output and interpreter exit without pretending that those costs were individually observed. Equivalent instrumentation is applied to both arms.

Table 5. Newly measured original-design replay, seconds. Exclusive stages sum with residual to the parent whole process; no training warm-up block is excluded.

Stage	Separate reference	Separate-built cache
cache persist verify read	0.000	0.235
checkpoint write	1.058	1.072
final evaluation	231.607	114.291
final generation	41.164	40.507
final identity validation	2.334	1.560
initial evaluation	262.579	116.998
initial fixture identity	0.753	0.763
initial generation	47.714	40.821
load data tokenizer policy validate	8.103	8.197
measured training	793.553	497.835
optimizer rng restore	0.008	0.008
reference construction and identity	1.241	1.198
reference precompute test	0.000	60.512
reference precompute train	0.000	228.436
reference precompute validation	0.000	55.894
shutdown release	0.200	0.178

Table 5. Newly measured original-design replay, seconds. Exclusive stages sum with residual to the parent whole process; no training warm-up block is excluded.

Stage	Separate reference	Separate-built cache
Unallocated parent residual	5.212	4.903
Parent whole process	1395.527	1173.407

In this new replay, whole-process times are 1395.527 s versus 1173.407 s, a signed reduction of 15.92%. Training itself takes 793.553 s versus 497.835 s. The breakdown explains these new observations, not the unrecorded split of older costs. Absolute runtime can move with concurrent system activity, thermal state and ordinary variation; exclusivity was not established.

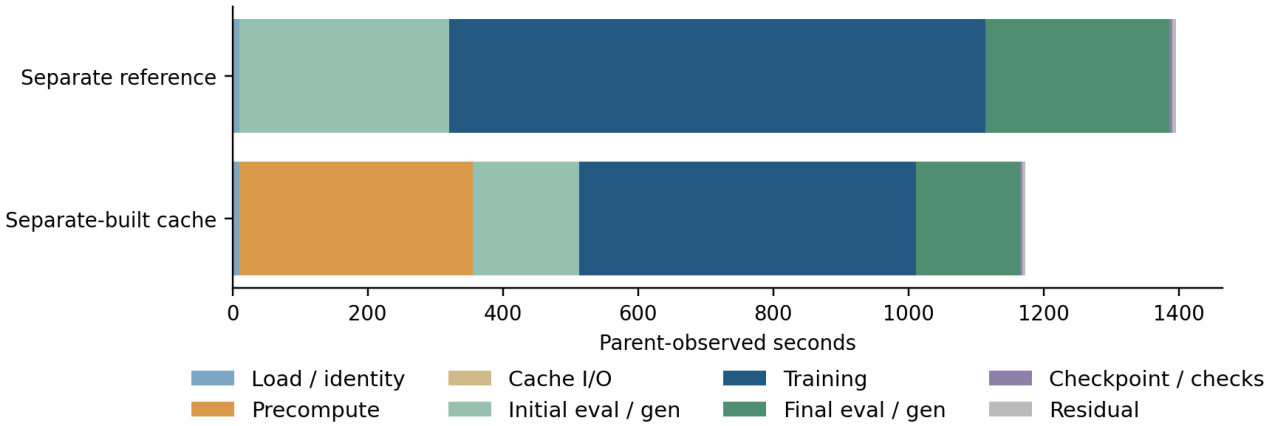


Figure 3. New original-design learning replay. Cache construction, evaluation/generation and training are separate; the parent residual is shown explicitly. These are single paired workflow executions.

$$T_u(E) = S_u + EU, \qquad T_c(E) = S_c + P + EC$$

$$E > \frac{S_c - S_u + P}{U - C} \quad \text{when} \quad U > C$$

Here S_c explicitly excludes reference precomputation P. The model assumes an unchanged reference and tokenized population, reusable cache, stationary per-pair cost and no evaluation/generation or full learning checkpoint. If U is no greater than C, extra passes do not yield a finite amortization point under these assumptions. Setup, cold construction and cache reuse are never charged twice. Projections describe these assumptions; they are not executed additional epochs.

Using the new B/C medians gives U=722.220, C=487.981, P=228.976, S_u=15.057 and S_c=15.118 seconds. The projected break-even is 0.978 passes, and projected cached-minus-shared time for one pass is -5.202 s. The setup proxy includes measured nontraining identity/persistence checks and the unallocated launch/exit residual, excludes benchmark warm-up and P, and should not be confused with the learning-workflow boundary.

9. Single-seed held-out outcomes

Table 6. Historical single paired seed, 16 updates; both reference strategies give identical saved per-example outcomes.

Split	Initial loss	Final loss	Positive margins	Ties
validation	0.693147	0.690077	42/64 (65.62%)	0/64
test	0.693147	0.690726	39/64 (60.94%)	0/64

Strict positives use beta*m > 1e-6; ties use |beta*m| <= 1e-6. Initial policy/reference equality gives 64 ties in each held-out split, not evidence of zero language ability. After training, test

positives are 39/64 and validation positives 42/64, with no ties. Initial/final losses are pair-weighted, not token-weighted. Double recomputation from stored scores supports the displayed rounded values. The paired learning trajectories and final outcomes agree for the single paired seed; this is not an average across seeds.

The eight reserved exploratory generations use greedy argmax and at most 24 new tokens, ending earlier at the assistant end token or remaining context cap. They are short examples without completed human ratings. No paid judge, preference survey or broad helpfulness/truthfulness study was run. The modest DPO-loss change and margin counts do not demonstrate reduced hallucination, improved general usefulness or a human-preference gain. Test outputs were examined during the historical study; this follow-up does not treat them as an untouched confirmatory evaluation.

10. Limits, reproducibility and responsibility

This is one small pretrained model, one filtered short-response population, one learning seed and one nonexclusive CPU host. Three timing trials describe this workload and cannot establish statistical significance, between-seed uncertainty or GPU behavior. AI-feedback preference labels can contain noise and systematic bias, and pretraining overlap is unknown. Stronger language-quality claims would require different evidence. Cached scores are valid only while the fixed reference, tokenizer/template, IDs/masks, precision/backend and other declared identity fields remain compatible. Numerical equivalence on this hardware is not a promise of bitwise portability across versions or devices.

A genuinely fresh project-local environment and scoped source copy were executed. The clean core suite passed 19 tests with no failures/errors/skips. Installed dependency consistency and file RECORD hashes were checked, user/system site packages were excluded, and imports resolved inside the clean environment. Cached pinned package payloads and hash-verified public model/data inputs were reused offline. Pinned tiny and pretrained TRL checks, the three-strategy short trajectory, toy resume coverage, numeric raw-evidence verification and paper/table regeneration ran there. The complete timing and learning campaigns were not repeated in that environment.

The implemented entry point is python scripts/finalize.py --offline --max-experiment-hours 6 --resume. --report-only regenerates manuscript and tables from evidence without training or downloads; --verify-only recalculates numeric checks without launching experiments. Existing raw evidence, immutable plans, result hashes and worker-source identities gate reuse. A fresh acquisition route remains python scripts/bootstrap.py --profile compact; use --offline only when all locked packages and public inputs are already available. Lightweight local tests are python -m pytest -q after installing the pinned environment. Exact Windows commands and the clean scope are recorded in the finalization report.

The available historical measured process total is 15073.360 s (4.187 h); with documented allowances it is 17055.114 s. This additional pass currently records 13466.732 s (3.741 h) of leaf subprocess time, including failed attempts and reproduction checks. Parent waits are not added again. Actual elapsed wall time, storage and the finalized ledger are reported separately. The additional limit is six experiment-process hours, ninety minutes per child, five GiB of downloads and twelve GiB additional storage. No new model/data payloads were downloaded.

The review archive contains source, tests, locked requirements, documentation, the synchronized report and numeric evidence sufficient to regenerate tables and audit scores. It excludes model weights, giant checkpoints, environments, caches, full datasets, credentials and unrelated material. Bundle verification checks the included evidence; it cannot reopen excluded checkpoint tensors or retrain offline without separately obtained pinned inputs. Local checkpoint audits and their exact scope are retained as evidence. Upstream licenses/notices are referenced without inventing a license grant for this project or third-party data.

The supported conclusion is that fixed-reference strategies can preserve the observed local trajectory while changing measured costs, and that a shared-base baseline and complete

accounting are necessary for interpreting those costs. The observed FP32 upstream discrepancy remains explicitly qualified.

Appendix A. Per-trial values and audit trail

Table A1. Historical paired-trial runtime reductions; these are not the ratio of medians.

Trial	Training reduction, %	Cold process reduction, %
0	38.132	19.844
1	25.141	8.369
2	30.185	11.919

Table A2. New measured per-trial values. All cold processes include setup; training excludes five warm-up updates.

Trial	Arm	Training, s	Whole process, s
0	A: separate reference	901.831	1159.711
0	B: shared base	907.790	1159.711
0	C: cold shared cache	611.667	1013.617
1	B: shared base	896.044	1142.434
1	C: cold shared cache	609.977	1013.588
1	A: separate reference	902.195	1151.478
2	C: cold shared cache	609.702	1014.831
2	A: separate reference	897.240	1145.515
2	B: shared base	902.774	1147.964

Table A3. New paired-trial reductions, 100*(baseline-comparison)/baseline. Negative values indicate slower execution.

Trial	Comparison	Training, %	Whole process, %
0	separate to shared	-0.661	-0.000
0	shared to cached	32.620	12.597
0	separate to cached	32.175	12.597
1	separate to shared	0.682	0.785
1	shared to cached	31.926	11.278
1	separate to cached	32.390	11.975
2	separate to shared	-0.617	-0.214
2	shared to cached	32.463	11.597
2	separate to cached	32.047	11.408

Local evidence roots are artifacts/finalization/<pass-id>/audit, parity, diagnose, precision, runs and evidence. inventory.json records the preserved source and historical files; frozen-source.zip and plan.json identify the follow-up. Each run records a spec, provenance, progress, metrics and parent-timing file. claims.json maps report tables to source hashes, fields, selection rules and formulas. The portable evidence manifest hashes included numeric files. Earlier failed/interrupted attempts remain local and visible in the ledger; no completion is inferred from a directory name alone.

References

[R1] Rafailov, Rafael; Sharma, Archit; Mitchell, Eric; Ermon, Stefano; Manning, Christopher D.; Finn, Chelsea (2023). Direct Preference Optimization: Your Language Model is Secretly a Reward Model. Retrieved 2026-10-15. Version/revision: arXiv:2305.18290v3. Primary source.

[R2] Eric Mitchell and contributors. Original DPO implementation. Retrieved 2026-10-15. Version/revision: f8b8c0f49dc92a430bae41585f9d467d3618fe2f. Primary source.

[R3] Hugging Face. TRL DPO Trainer documentation. Retrieved 2026-10-15. Version/revision: 0.24.0. Primary source.

[R4] Hugging Face. TRL DPO implementation v0.24.0. Retrieved 2026-10-15. Version/revision: v0.24.0; content hash pinned. Primary source.

[R5] Qwen Team. Qwen3-0.6B model card. Retrieved 2026-10-15. Primary source.

[R6] Hugging Face TB. SmolLM2-135M-Instruct model card. Retrieved 2026-10-15. Primary source.

[R7] Hugging Face H4. UltraFeedback Binarized dataset card. Retrieved 2026-10-15. Primary source.

[R8] Hugging Face. Chat templates. Retrieved 2026-10-15. Version/revision: 4.57.1. Primary source.

[R9] PyTorch. PyTorch benchmarking. Retrieved 2026-10-15. Version/revision: PyTorch tutorials 2.14.0+cu130 (documentation only). Primary source.

[R10] PyTorch. PyTorch profiler. Retrieved 2026-10-15. Version/revision: PyTorch tutorials 2.14.0+cu130 (documentation only). Primary source.

[R11] PyTorch. PyTorch reproducibility. Retrieved 2026-10-15. Version/revision: 2.9. Primary source.

[R12] Hugging Face. PEFT LoRA documentation. Retrieved 2026-10-15. Version/revision: 0.17.0 documentation; installed PEFT 0.17.1 API smoke checked. Primary source.

[R13] Hu, Edward J.; Shen, Yelong; Wallis, Phillip; Allen-Zhu, Zeyuan; Li, Yuanzhi; Wang, Shean; Wang, Lu; Chen, Weizhu (2021). LoRA: Low-Rank Adaptation of Large Language Models. Retrieved 2026-10-15. Primary source.

[R14] Cui, Ganqu; Yuan, Lifan; Ding, Ning; Yao, Guanming; He, Bingxiang; Zhu, Wei; Ni, Yuan; Xie, Guotong; Xie, Ruobing; Lin, Yankai; Liu, Zhiyuan; Sun, Maosong (2023). UltraFeedback: Boosting Language Models with Scaled AI Feedback. Retrieved 2026-10-15. Primary source.

[R15] Hugging Face. PEFT model adapter API. Retrieved 2026-10-15. Version/revision: Documentation 0.17.0; installed PEFT 0.17.1 disable_adapter implementation inspected locally. Primary source.